

Python Fundamentals

This document contains useful information about Python fundamentals that will be used throughout this course.

Note: The terminology included in this document and a basic understanding of the material below is course content therefore it may be included on exams or quizzes.

Table of Contents

Introduction to Python	3
Terminology	3
Variables and Expressions	4
Operators	6
Arithmetic Operators	6
Relational Operators	7
Logical Operators	7
Assignment Operators	8
Special Operators	9
Functions	10
Lambda Functions	10
Conditionals	11
if	11
else	11
elif	12
one-line conditional expression	13
Iteration	14
While Loop	14
For Loop	15
Range	15
Loop Control Statements	16
Strings	17
Concatenation	18
Multiplication	18
Indexing and Slicing	18
String Methods	19
f-strings	20
Lists	20
Addition and Multiplication	20
Indexing and Slicing	21
sorted() vs .sort()	21
List Methods	23
Tuples	23
Addition and Multiplication	24
Indexing and Slicing	24
Tuple Methods	25

Dictionaries	25
Accessing Values	26
Updating Dictionaries	26
Dictionary Methods	28
Sets	28
Adding and Removing Elements	28
Operations	29
Comprehensions	30
List Comprehensions	30
Dictionary Comprehensions	30
Casting to Change Data Types	33
Implicit Type Conversion:	33
Explicit Type Conversion (Casting):	33
Handling Potential Loss of Information:	34
Error Handling with Casting:	34
Modules	34
Modules vs Scripts	35
if <code>__name__ == "__main__"</code>	35
Example of running .py file as Module / Script	37
Try and Except	37
References	38

Introduction to Python

Python is an interpreted, object-oriented, high-level programming language with dynamic data typing. Its high-level built-in data structures and dynamic typing makes it very useful as a scripting language. It's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are open-source so they can be freely distributed.

In order to begin manipulating data, we need to establish common ground regarding the terminology used in this course.

Terminology

Data Type - An object's type defines the possible values and operations that type supports.

Variable - A container for storing a data value. A Python variable is created the moment you first assign a value to it.

Operator - A construct that can manipulate the value of operands. Python operators include the following types: Arithmetic Operators, Comparison (Relational) Operators, Assignment Operators, and Logical Operators

Expression - An instruction that combines values and operators and evaluates down to a single value

Function - A block of code that only runs when it is called

Iterable - A Python object capable of returning its members one at a time, permitting it to be iterated over in a for-loop

Mutable - An object that is able to be changed after it is created

Immutable - An object that is not able to be changed after it is created

Subscriptable - Objects that are "containers", meaning they contain or store other objects and data. Examples of subscriptable data structures include; strings, lists, tuples, and dictionaries.

Variables and Expressions

Variables are identifiers for a value or an object. In order to establish the variable name, we can use the assignment operator, `=`. To demonstrate this, let's assign the value `2020` to a variable named `year`.

```
>>> year = 2020
```

There are some rules that are important to keep in mind when naming your variables. All variable names must contain some combination of alphanumeric characters (A-z, 0-9) or an underscore. Variable names cannot start with a number and are case sensitive. Thus,

```
>>> year = 2020
>>> Year = 2020
>>> year_ = 2020
```

are all three different variables, while the following variable declaration would be invalid.

```
>>> 2020year = 2020
```

It is conventional to choose a logical name for a variable that describes that value it holds. Although `x`, `y`, `a`, `_`, etc. are valid variable names they may not describe the value contained within the variable. In a production environment where code is shared, it is beneficial to your coworkers and yourself to choose descriptive variable names.

Now that we know how to assign a value to a variable, let's perform some operations in an expression. An **expression** is an instruction that combines values and operators and always evaluates down to a single value. To show an example of an expression, suppose we wanted to increment the value of `year` by 1. The following line of code uses the addition operator to add `1` to the value stored in the `year` and returns the result. Notice that the value of `year` did not change.

```
>>> year + 1
2021
>>> year
2020
```

In the example above, `year + 1` is an expression because the result evaluates down to a single value, in this case, `2021`. If we wanted to override the current value assigned to `year` we must use the assignment operator.

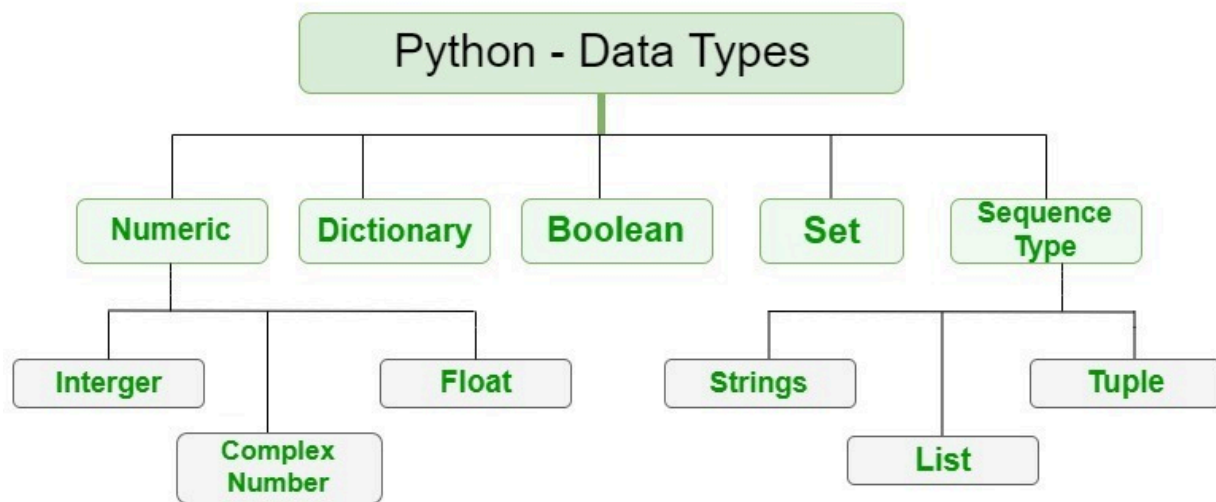
```
>>> year = year + 1
>>> year
2021
```

Note that when using the assignment operator, the code on the right of the operator *must* be an expression. Although the syntax above is sufficient, there is a condensed and equivalent way to perform the same operation.

```
>>> year = 2020
>>> year += 1
>>> year
```

2021

In Python, a variable's type is the data type of the value that the variable currently holds. Therefore, Python is dynamically typed.



To recap this section, we've shown how to:

1. define a variable
2. perform simple operations
3. use an expression
4. override a variable's value

The same concepts scale using other operators, which will be discussed in the next section.

Operators

In native Python there are six operator categories: arithmetic, relational, logical, bitwise, assignment, and special. Below are explanations and examples of each category.

Arithmetic Operators

Arithmetic operators are used to perform operations like addition, subtraction, multiplication, division, etc. Below is a table of arithmetic operators.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Floor Division
%	Modulus
**	Exponent

The result that the operator yields depends on the types used with the operator. For example,

```
>>> 2 + 2
4
```

Reasonably returns 4 because the type of each value is `int`, however

```
>>> "2" + "2"
"22"
```

Returns a value of "22" because the type of each value is `str` and not `int`. Since the value of each type is `str`, Python performs string concatenation. Python defines the acceptable use of each operator using method overloading. We might think that,

```
>>> "22" - "2"
```

```
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

May remove any instance of "2" from a string, but it doesn't because the developers that wrote Python's source code did not define an arithmetic operator to do so. Instead, we get a type error because the - operator is not supported for strings.

Relational Operators

Relational operators are used to perform comparisons.

Operator	Descriptions
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to

Logical Operators

There are three logical operators: **and**, **or**, and **not**. The meaning of these operators is similar to their meaning in English. For example:

```
>>> x > 0 and x < 10
```

is true only if x is greater than 0 and at the same time, x is less than 10. How would you describe this in words? You would say that x is between 0 and 10, not including the endpoints.

```
>>> n - 2 == 0 or n - 3 == 0
```

is True if either of the conditions is True, that is if n is equal to 2 or 3. In this case, one, or the other, or both of the parts has to be True for the result to be True.

Finally, the not operator negates a Boolean expression, so

```
>>> not x > y
```

is True if $x > y$ is False, that is, if x is less than or equal to y.

There is a very common syntax mistake that programmers make when writing boolean expressions. In Python, the expression `number == 5 or 6 or 7` is incorrect. The **or** operator must join the results of three equality checks. The correct way to write this is `number == 5 or number == 6 or number == 7`. This may seem like a lot of typing but it is absolutely necessary. You cannot take a shortcut.

Logical Operators are performed after Arithmetic Operators and Relational Operators in the order of **not** first, then **and**, and lastly **or**.

Assignment Operators

Assignment statements create new variables and also give them values to refer to. The **assignment token**, `=`, should not be confused with *equality* (we will see later that equality uses the `==` token). The assignment statement links a *variable*, on the left-hand side of the operator, with a *value*, on the right-hand side.

```
>>> message = "What's up, Doc?"
>>> n = 17
>>> pi = 3.14159
```

The **assignment token**, `+=` : adds the value found in the right operand to the value found in the left operand.

```
>>> x = 5
>>> x += 3
>>> x
8
```

The **assignment token**, `-=` : subtracts the value of the right operand from the value found in the left operand.

```
>>> x = 5
>>> x -= 3
2
```

The **assignment token**, `*=` : multiplies the value of the right operand by the value of the left operand.

```
>>> x = 5
>>> x *= 3
>>> x
15
```

The **assignment token**, `/=` : divides the value of the left operand by the value of the right operand.

```
>>> x = 5
>>> x /= 3
>>> x
1.667
```

The **assignment token**, `%=` : divides the value found in the left operand by the value found in the right operand. The remainder will be placed in the left operand.

```
>>> x = 5
>>> x %= 3
>>> x
2
```

The **assignment token**, `**=` : determines the exponential value found in the left operand when raised to the power of the value in the right operand. Example:

```
>>> x = 5
>>> x **= 3
>>> x
125
```

The **assignment token**, `//=` : uses floor division to divide the value found in the left operand by the value of the right operand. The integer result will be placed in the left operand. Example:

```
>>> x = 5
>>> x //= 3
>>> x
1
```

Special Operators

The `'is'` operator is an identity operator used to determine whether a value is of a certain class or type. It evaluates to true if the variables on either side of the operator point to the same object and false otherwise.

```
>>> 6 is 6
True
>>> type("hello world") is str
True
```

The `'in'` operator is a membership operator used to check if a value exists in a sequence or not. It tests for membership in a sequence, such as strings, lists, or tuples.

```
>>> "a" in "apple"
True
>>> 9 in [1, 2, 5, 9, 13, 7]
True
```

Functions

In Python, a function is a group of related statements that performs a specific task. Functions are used to divide programs into manageable pieces and to create reusable code.

The syntax for a Python function is

```
def function_name (parameters):  
    ...     statements
```

A function is called by using its name and including arguments that correspond to the parameters.

```
>>> def add (num1, num2):  
...     return num1 + num2  
>>> print(add (2, 3))  
5
```

Functions who do not explicitly return a value by including a return statement, implicitly return the value `None` a special value of type `NoneType`.

Lambda Functions

Another way to define a function is by using Python's `lambda` reserved word. For example, we can create an add function identical to the one above using the following syntax:

```
>>> add = lambda num1, num2 : num1 + num2  
>>> print(add(2, 3))  
5
```

Notice the parameters are defined after `lambda` and before the colon, while the returned value is defined in the expression following the colon. Oftentimes lambda functions are used to perform a simple task which we will revisit in the sorting section below. Lambda functions can also be anonymous functions, meaning that they are created without assigning a name.

Conditionals

In Python, we can use conditional statements to determine whether a certain condition exists before an action should be performed. Conditional statements help improve the efficiency of your code by allowing you to control when you want a particular block of code to execute. For example, you can use conditional statements to check whether a certain variable exists or multiple criteria are met before executing code. There are 3 main statements that you will use when creating conditionals in Python.

if

When creating conditional statements, you will always begin with the `if` statement followed by the **condition**, which is a Boolean expression. The structure of a basic conditional statement using `if` is as follows:

```
>>> if condition:
...     code to execute if condition is True
```

Notice that the code we want to execute if the condition is met is placed directly under the `if` statement and is indented. Additionally, the code will also only execute if the conditions expression evaluates to `True`. If the condition evaluates to `False`, the code contained under the indented block will be skipped and the next unindented block of code will run.

Here is an example:

```
>>> x = 2
... if x % 2 == 0:
...     print(str(x) + " is an even number!")
... print("Finished executing!")
"2 is an even number!"
"Finished executing!"
```

Notice in our example, we first set the variable `x` to the int `2`. We then create a conditional statement checking whether `x % 2 == 0` evaluates to `True` or `False`. Since `2 % 2 == 0` is `True`, the line of code `print(str(x) + " is an even number!")` is executed. Finally, the statement `print("Finished executing!")` is always executed at the very end, as it is not a part of the conditional statement. Now say we set `x = 3` and rerun the conditional statement. This time, since `x % 2 == 0` evaluates to `False`, the indented line `print(str(x) + " is an even number!")` is **not** executed. Instead, the line `print("Finished executing!")` will only be executed.

else

When using conditional statements, there are times where we want another block of code only to run when the condition Boolean expression evaluates to `False`. We can accomplish this by using the `else` statement. The structure of conditional statements including `if` and `else` is as follows:

```
>>> if condition:
...     code to execute if condition is True
... else:
...     code to execute if condition is False
```

Notice that the `if` and corresponding `else` statements are on the same indentation level.

We are now able to extend our conditional statement to handle two different scenarios: if the condition is `True`, the indented code beneath the `if` statement will be executed whereas if the condition is `False`, the indented code beneath the `else` statement will be executed.

For an example, we have:

```
>>> course = "2316"
>>> if course == "2316":
...     print("I am in the right class!")
... else:
...     print("This is not my class...")
"I am in the right class!"
```

For this example, we set the variable `course = "2316"`. Our condition is `course == "2316"`, and since this Boolean expression evaluates to `True`, the line `print("I am in the right class!")` is executed. Notice that the print statement under the `else` statement is not executed at all, since the `else` statement will only be executed if the above condition Boolean expression evaluates to `False`. Thus, you will never have both the `if` and corresponding `else` statements executing together. If we set the variable `course = "1331"` instead, the `print("This is not my class...")` will be the only statement to be executed instead.

elif

At times, we may need to handle more than two possibilities within our code. Thus we will require more than two branches. One way we can express a computation for this is to use chained conditionals. To create chained conditionals, we can use the `elif` statement. Here is a simple conditional statement structure utilizing the `elif` statement:

```
>>> if first_condition:
...     print("The first condition passed!")
... elif second_condition:
...     print("The first condition failed... but the second condition passed!")
... else:
...     print("Both conditions failed...")
```

Note that you may include multiple `elif` statements in your chained conditional statements. First, we start off with the `if` statement, including the first conditional. If the conditional paired with the `if` statement evaluates to `True`, then Python will not run the

other `elif` and `else` statements. However, if the `if` statement conditional evaluates to `False`, Python will check the next `elif` statement conditional. Python will keep checking the next `elif` statement conditional until one of them evaluates to `True` or it hits the `else` statement instead. Once an `elif` statement conditional is met, Python will break out of the chained conditional block and move on to the next line. Let's illustrate an example:

```
>>> x = 3
... if x > 4:
...     print("I am greater than 4")
... elif x < 2:
...     print("I am less than 2")
... elif x == 3:
...     print("I am 3!")
... else:
...     print("Did not meet any conditions")
"I am 3!"
```

We set `x = 3`. Since `x > 4` evaluates to `False`, we move to the first `elif` statement. Since `x < 2` is `False`, we move to the next `elif` statement. Since `x == 3` is `True`, we see that the print statement under `elif x == 3` is executed. Since the `elif` statement is executed, Python breaks out of the chained conditional statement and does not execute the `else` statement. Now suppose `x = 5`. The `if x > 4` evaluates to `True`, and the print statement under this condition is executed. Python then breaks out of the chained conditional statement.

one-line conditional expressions

Occasionally, we encounter situations where we need a concise way to make decisions and express them within a single line of code. Python offers a solution with its one-line conditional expressions. These compact constructs allow us to achieve a streamlined decision-making process. Here is the basic syntax of a one-line conditional expression:

```
>>> result_if_true if condition else result_if_false
```

In this construct, `condition` represents the condition you want to check. If `condition` evaluates to `True`, then `result_if_true` is returned; otherwise, `result_if_false` is returned. Let's illustrate it with an example:

```
>>> x = 10
... message = "Greater than 5" if x > 5 else "Less than or equal to 5"
... print(message)
```

In this example, we set `x = 10`. The one-line conditional expression checks if `x > 5`. Since `if x > 5` evaluates to `True`, the value "Greater than 5" is assigned to the 'message' variable. When we print 'message', it displays "Greater than 5".

Iteration

Oftentimes, we will want to create repetitive tasks while coding in Python. Instead of duplicating multiple lines of code in Python, we can simplify our code to handle multiple iterations of the same tasks with more condensed code. The two most common statements are `for` and `while` loops.

While Loop

One way to iterate over a block of code is to use the `while` statement. We usually use while loops when we do not know the number of times we want to iterate. Thus, we want to use a loop that iterates until a condition is met. This is because the `while` in Python is used to iterate over a block of code as long as the conditional expression evaluates to `True`. Here is the general syntax:

```
>>> while condition:
...     code that runs while condition is True
```

Because while loops continue to run until the conditional expression evaluates to `False`, you could encounter an infinite iteration loop. To avoid this, Python users include a variable that updates the conditional expression for each iteration. Here is an example of adding a counter to update the condition for a while loop:

```
>>> countdown = 5
... while countdown > 0:
...     print(str(countdown) + "...")
...     countdown -= 1
... print("Blastoff!")
"5..."
"4..."
"3..."
"2..."
"1..."
"Blastoff!"
```

As you may see, our conditional expression for the while loop is `countdown > 0`. While this condition evaluates to `True`, the indented block of code contained within the while loop will always execute. Each time the indented block of code runs, the line `countdown -= 1` decrements the `countdown` variable. This allows the conditional expression `countdown > 0` to eventually evaluate to `False`. Once the conditional expression evaluates to `False`, you will break out of the while loop and continue down to the next line. For example, if we set `countdown = 1`, we will perform one total iteration of the while loop before terminating the while loop.

For Loop

There will be times where we want to loop through a range of values with a fixed length, such as a list of integers or dictionaries of keys and values. Since we have a more general sense of how many iterations we want to perform, we can utilize a definite loop using the for statement. Thus, using a for loop is the same as looping through a known set of items, allowing us to run through as many iterations as there are items in that specific set. Here is the general syntax:

```
>>> for item in set:
...     code to execute
```

Unlike while loops, you will most likely never encounter the possibility of creating an infinite loop of iterations. Let's see an example of for loops:

```
>>> a_list = ["Bohong", "Marylyn", "Manny"]
>>> for name in a_list:
...     print(name + " is a returning TA!")
"Bohong is a returning TA!"
"Marylyn is a returning TA!"
"Manny is a returning TA!"
```

In this for loop, we assign each individual string within `a_list` to the variable name for each iteration. In the first iteration, the string `"Bohong"` is assigned to `name`, `"Marylyn"` is assigned to `name` in the second iteration, and `"Manny"` is assigned to `name` in the third and final iteration. Since the string `"Manny"` is the final item within `a_list`, the for loop ends once all indented code is executed when `"Manny"` is assigned to `name`.

Range

`range()` is a built-in Python function that creates an object containing values that are commonly iterated over using a for loop. The `range()` function allows users to generate a sequence of numbers that fit a certain pattern. Depending on the number of parameters passed into `range()`, users can decide where that sequence of numbers will begin and end, as well as what difference is between each number. The general syntax is outlined below:

`range(start, stop, step)`

- `start`: integer stating where the sequence of numbers should start at (inclusive). Default is 0
- `stop`: integer stating where the sequence of integers should stop at (exclusive)
- `step`: integer stating the increment between each integer in the sequence. Default is 1

Here are some examples:

```
>>> for number in range(2):
...     print(number)
0
```


1

```
>>> for number in range(3, 10, 3):  
...     print(number)
```

3

6

9

```
>>> a_list = ["Bohong", "Marylyn", "Manny"]  
>>> for index in range(len(a_list)):  
...     print(index)
```

0

1

2

```
>>> a_list = ["Bohong", "Marylyn", "Manny"]  
>>> for index in range(len(a_list)):  
...     print(a_list[index])  
"Bohong"  
"Marylyn"  
"Manny"
```

Loop Control Statements

Loop control statements change execution from its normal sequence. Following are the loop control statements provided by Python:

break: Break statements in Python are used to bring the control out of the loop when some external condition is triggered.

```
>>> if x in word:  
...     break
```

continue: Continue statements are opposite to that of break statement; instead of terminating the loop, it forces the loop to execute the next iteration of the loop without any of the succeeding code being executed.

```
>>> if x in word:  
...     continue
```

pass: Pass statements are used to ignore a particular block of code like loops, conditionals, functions, and classes.

```
>>> if x in word:
...     pass
```

Strings

In Python, strings are arrays of bytes that represent unicode characters. To denote strings in Python, string literals are surrounded by either single or double quotation marks. Thus, "2316" is the same as '2316'. You can further display a string literal by using the print statement, such as print("Welcome to CS 2316"). You can also create docstrings in Python, which are documentation strings that have a similar function as comments made in Python. Docstrings also allows users to create multiline comments, instead of declaring a pound # at the beginning of each line. To denote a docstring, surround the following block of text you want to comment out with triple quotes (""" or '''). Let's see an example:

```
>>> """
... This is a
... really
... long
... docstring
... """
```

In Python, case-sensitivity matters for string literals. For example, saying "cs2316" is **not** the same as saying "CS2316".

Strings are also immutable, meaning they cannot be changed after being created. For example, the following would error:

```
>>> a_str = "Hello World!"
>>> a_str[0] = "h"
TypeError: 'str' object does not support item assignment
```

Just like how you can iterate with lists, tuples, and dictionaries, you can also iterate through strings using either a for or while loop.

```
>>> a_str = "apple"
>>> for letter in a_str:
...     print(letter)
"a"
"p"
"p"
"l"
"e"
```

Concatenation

There will be times where you will want to merge or combine multiple strings together, otherwise known as concatenating strings. In Python, it is actually really simple to concatenate strings together. In order to merge two or more strings together, you will need to use the “+” operator.

```
>>> a_str = "Marylyn"
>>> b_str = " Chen"
>>> a_str + b_str
"Marylyn Chen"
```

Multiplication

Although it sounds strange, Python allows you to do a version of multiplication with strings. When you multiply a string by an integer, Python returns a new string. The returned string is the original string, repeated X number of times (where X is the integer). You can only multiply strings with integers, and operations such as ** will error.

```
>>> a_str = "apple"
>>> a_str * 3
"appleappleapple"
```

Indexing and Slicing

What if we wanted to return a specific range of characters from a string? Python allows you to use indexing and slicing to return specific parts of strings you want to access. To index specific positions, or index, of the letters of in string, you will use this notation: `string[int]`. The `[int]` represents the position of the letter you wish to access. If a negative value is used as the index, Python will count backwards from the last index in the string having the value of -1.

```
>>> a_str = "apple"
>>> a_str[3]
"l"
>>> a_str[-2]
"l"
>>> a_str[5]
IndexError: string index out of range
```

To slice specific portions of the string, the general notation you will use is `string[start:stop:step]`. Start represents the index you wish to start with (inclusive), stop represents the index you wish to end at (exclusive), and step is the increment of indices between the previous and the next indices. All values can be positive or negative.

```
>>> a_str = "python"
>>> a_str[0: 3]
```

```

"pyt"
>>> a_str[1:4:2]
"yh"
>>> a_str[::-1]
"nohtyp"
>>> a_str[-1:-3:-1]
"no"

```

String Methods

Method	Usage
<code>.index()</code>	Returns the first occurring index of that character parameter. Errors if no match found
<code>.lower()</code>	Converts a string into lower case and returns the new string
<code>.upper()</code>	Converts a string into upper case and returns the new string
<code>.isalpha()</code>	Returns True if all characters in the string are in the alphabet, else returns False
<code>.isdigit()</code>	Returns True if all characters in the string are digits, else returns False
<code>.isupper()</code>	Returns True if all characters in the string are upper case, else returns False
<code>.islower()</code>	Returns True if all characters in the string are lower case, else returns False
<code>.join()</code>	Returns a new string containing all the items in an iterable joined together by a separator string.
<code>.split()</code>	Returns a new list that is formed by splitting the string by a separator string (default is space)
<code>.replace()</code>	Replaces a specified phrase with another specified phrase and returns the new string

f-strings

Also called “formatted string literals,” f-strings are string literals that have an “f” at the beginning and curly braces containing expressions that will be replaced with their values. The expressions are evaluated at runtime and then formatted using the `__format__` method of the string class.

```
>>> a_str = "apple"
>>> print(f"{2} plus {3} equals {2 + 3}")
"2 plus 3 equals 5"
```

Lists

One of the various data structures found in Python are **lists**. Lists are mutable objects that can hold a collection of elements within. They are denoted by `[ ]`, and each element within the list are separated by commas. Lists can hold any number of elements, where each element may be different types (integers, floats, strings, tuples, etc.). Lists may also contain other lists within them, which is called a nested list. To declare a list, you can use the square brackets `[ ]` or Python’s built-in `list()` function.

```
>>> a_list = [1, 2.5, "3"]
>>> a_list
[1, 2.5, "3"]
>>> a_tup = (1, 2, 3)
>>> b_list = list(a_tup)
>>> b_list
[1, 2, 3]
```

Just like how you can iterate with strings, tuples, and dictionaries, you can also iterate through lists using either a `for` or `while` loop.

```
>>> ta_list = ["Bohong", "Marylyn", "Manny", "Adam", "Grace"]
>>> for ta in ta_list:
...     print(ta)
"Bohong"
"Marylyn"
"Manny"
"Adam"
"Grace"
```

Addition and Multiplication

Python allows you to use addition and multiplication operators on lists. When the “+” operator is used on two or more separate lists, Python combines all the elements within the lists into one list and returns the new list. When you multiply a list with an integer while

using the “*” operator, Python will repeat the elements within the list X number of times (where X is the integer).

```
>>> a_list = [1, 2, 3]
>>> b_list = [4, 5, 6]
>>> a_list + b_list
[1, 2, 3, 4, 5, 6]
>>> c_list = [0, 1, 2]
>>> c_list * 3
[0, 1, 2, 0, 1, 2, 0, 1, 2]
```

Indexing and Slicing

There will be times where you want to access or return the specific portions of a list. In order to do so, you can perform list indexing and slicing to retrieve the parts of the list you want to access.

To index specific positions, or index, of the elements in a list, you will use this notation: `list[int]`. The `[int]` represents the position of the element you wish to access. Once you find the index of the element you wish to access, you can also set the index to a new element.

```
>>> a_list = ["a", "b", "c"]
>>> a_list[1]
"b"
>>> a_list[1] = "B"
>>> a_list
["a", "B", "c"]
```

To slice specific portions of the list, the general notation you will use is `list[start:stop:step]`.

`start` represents the index you wish to start with (inclusive), `stop` represents the index you wish to end at (exclusive), and `step` is the increment of indices between the previous and the next indices.

```
>>> ta_list = ["Bohong", "Marylyn", "Manny", "Adam", "Grace"]
>>> ta_list[1::2]
["Marylyn", "Adam"]
>>> ta_list[::-1]
["Grace", "Adam", "Manny", "Marylyn", "Bohong"]
```

sorted() vs .sort()

Since lists are mutable data structures, we are able to modify the elements within a list if we want. One major modification to lists that programmers use is sorting the elements within a list. There are two main ways you can sort a list: using the function `sorted()` and using the method `.sort()`.

The `sorted()` function sorts the given list sequence in either ascending order or in descending order and **returns the sorted list**. This function does not affect the original list.

```
>>> a_list = [2, 3, 1, 6]
>>> sorted(a_list)
[1, 2, 3, 6]
```

`sorted()` defaults to sorting the element sequence in **ascending order**, but you can sort in **descending order** by adding `reverse = True` (default is `reverse = False`).

```
>>> a_list = [2, 3, 1, 6]
>>> sorted(a_list, reverse = True)
[6, 3, 2, 1]
```

There will be times where you will have a list with nested objects, such as other lists and tuples, within your main list. In order to sort based on a specific index of the nested objects, you can create a lambda function and pass it in with your list.

```
>>> a_list = [("apples", 3), ("bananas", 1), ("starfruit", 2)]
>>> sorted(a_list, key = lambda x: x[1]) # sorts by the first index
of each nested tuple
[("bananas", 1), ("starfruit", 2), ("apples", 3)]
```

The `.sort()` method is similar to `sorted()`. `.sort()` sorts the given list sequence in either ascending order or in descending order and **mutates the original list**. This method affects the original list and **returns None**.

```
>>> a_list = [2, 3, 1, 6]
>>> a_list.sort()
>>> a_list
[1, 2, 3, 6]
```

`.sort()` defaults to sorting the element sequence in **ascending order**, but you can sort in **descending order** by adding `reverse = True` (default is `reverse = False`).

```
>>> a_list = [2, 3, 1, 6]
>>> a_list.sort(reverse = True)
>>> a_list
[6, 3, 2, 1]
```

There will be times where you will have a list with nested objects, such as other lists and tuples, within your main list. In order to sort based on a specific index of the nested objects, you can create a lambda function and pass it in with your list.

```
>>> a_list = [("apples", 3), ("bananas", 1), ("starfruit", 2)]
>>> a_list.sort(key = lambda x: x[1]) # sorts by the first index of
each nested tuple
>>> a_list
[("bananas", 1), ("starfruit", 2), ("apples", 3)]
```

Both the `.sort()` method and the `sorted()` function allow you to sort using multiple keys. If your `lambda` function returns a tuple, both `.sort()` and `sorted()` will sort the list successively by each of the items in the tuple.

```
>>> a_list = [("apples", 1), ("bananas", 2), ("starfruit", 1)]
>>> a_list.sort(key = lambda x: (x[1],x[0])) # sorts first by count
and then by name
>>> a_list
[("apples", 1), ("starfruit", 1), ("bananas", 2)]
```

List Methods

Method	Usage
<code>.append()</code>	Adds an element at the end of the list
<code>.extend()</code>	Add the elements of a list (or any iterable) to the end of the current list
<code>.index()</code>	Returns the index of the first element with the specified value
<code>.count()</code>	Returns the number of elements with the specified value
<code>.remove()</code>	Removes the first item with the specified value

Enumerate(list)

In Python, `enumerate` is a built-in function which takes in a list as a parameter and returns an `enumerate` object. When you unpack this `enumerate` object (cast it to a list) the object is a list of tuples where the first index of each tuple (see directly below) is the index number of the original list, and the second index of the tuple is the value at that index of the list

Example:

```
alist = ['a', 'b', 'c']
print(list(enumerate(alist)) -> [(0, 'a'), (1, 'b'), (2, 'c')])
```


Tuples

In Python, tuples are very similar to lists. The main difference between the two is lists are mutable while tuples are **immutable**, meaning once the elements of a tuple are assigned, we cannot change those elements. This also means that we cannot sort tuples, as the indices of the elements within the tuple will stay preserved. Tuples are denoted with () and each element is separated by a comma and can hold various elements of different types (integers, floats, lists, strings, tuples, etc.). You can also use the built-in `tuple()` function to create new tuples.

Note: when creating tuples with only one element, you must still include a comma.

```
>>> tup = (1, 2, 3)
>>> tup
(1, 2, 3)
>>> a_list = ["a", "b", "c"]
>>> tuple(a_list)
("a", "b", "c")
>>> new_tup = ("apple", )
>>> new_tup
("apple", )
```

Just like how you can iterate with strings, lists, and dictionaries, you can also iterate through tuples using either a for or while loop.

```
>>> ta_tup = ("Bohong", "Marylyn", "Manny", "Adam", "Grace")
>>> for ta in ta_tup:
...     print(ta)
"Bohong"
"Marylyn"
"Manny"
"Adam"
"Grace"
```

Addition and Multiplication

Python allows you to use addition and multiplication operators on tuples. When the "+" operator is used on two or more separate tuples, Python combines all the elements within the tuples into one tuple and returns the new tuple. When you multiply a tuple with an integer while using the "*" operator, Python will repeat the elements within the tuple X number of times (where X is the integer).

```
>>> a_tup = (1, 2, 3)
>>> b_tup = (4, )
>>> a_tup + b_tup
(1, 2, 3, 4)
>>> c_tup = (0, 1, 2)
```

```
>>> c_tup * 3
(0, 1, 2, 0, 1, 2, 0, 1, 2)
```

Indexing and Slicing

You can also index and slice tuples, just like strings and lists.

To index specific positions, or index, of the elements in a tuple, you will use this notation: `tuple[int]`. The `[int]` represents the position of the element you wish to access.

```
>>> tup = ("a", "b", "c")
>>> tup[1]
"b"
```

```
>>> tup[1] = "B"
```

`TypeError: 'tuple' object does not support item assignment`

To slice specific portions of the tuple, the general notation you will use is `tuple[start:stop:step]`.

`start` represents the index you wish to start with (inclusive), `stop` represents the index you wish to end at (exclusive), and `step` is the increment of indices between the previous and the next indices.

```
>>> tup = (1, "2", 3.0, [4])
>>> tup[1:2]
("2", [4])
>>> tup[::-1]
([4], 3.0, "2", 1)
```

Tuple Methods

Method	Usage
<code>.count()</code>	Returns the number of times a specified value occurs in a tuple
<code>.index()</code>	Searches the tuple for a specified value and returns the position of where it was found

Dictionaries

A dictionary is a type of **unordered** data structure that consists of a collection of key-value pairs. When creating a dictionary, you must place the items within a set of curly braces `{ }` and separate with commas. Each item is composed of two parts: the key and the value. In a dictionary, every key is mapped to value. While dictionaries are mutable, the keys of the dictionary should be immutable (strings, integers, floats, or tuples with immutable elements) and must be **unique**. However, the values of the dictionary can be mutable and do

not have to be unique. To denote an item within a dictionary, you must use the following syntax: `key: value`. You can also create dictionaries with the built-in `dict()` function.

```
>>> a_dict = {"bear": "black", "parakeet": "green"}
>>> a_dict
{"bear": "black", "parakeet": "green"}
>>> a_list = [(90, "A"), (80, "B"), (70, "C")]
>>> dict(a_list)
{90: "A", 80: "B", 70: "C"}
```

Just like how you can iterate with strings, lists, and tuples, you can also iterate through dictionaries using either a `for` or `while` loop. Notice that we are only able to iterate through the keys of the dictionary!

```
>>> a_dict = {90: "A", 80: "B"}
>>> for key in a_dict:
...     print(key)
90
80
```

Accessing Values

Since dictionaries are unordered, you cannot index or slice to retrieve values. Instead, dictionaries use keys to access values that are stored within. To access a value with a key, the general syntax is `dictionary[key]`. However, note that you can only access the value that is mapped to the corresponding `[key]`. If you try to access a value with a key that is not present within the dictionary, you will raise a `KeyError`.

```
>>> a_dict = {"name": "Georgia Tech", "city": "Atlanta", "state":
"Georgia"}
>>> a_dict["city"]
"Atlanta"
>>> a_dict["zip_code"]
KeyError: "zip_code"
```

Updating Dictionaries

Since dictionaries are mutable, we are able to add and remove items to existing dictionaries or update existing values.

To update or add new items to a dictionary, you will use the `dictionary[key] = value` syntax. If the key is already present within the dictionary, the existing value gets updated. If the key is not present within the dictionary, a new item (key: value) pair is added to the dictionary.

```
>>> a_dict = {"name": "Georgia Tech", "city": "Atlanta", "state":
"Georgia"}
>>> a_dict["name"] = "Georgia State"
```

```
>>> a_dict["zip_code"] = 30303
{"name": "Georgia State", "city": "Atlanta", "state": "Georgia",
"zip_code": 30303}
```

We can also remove particular items from a dictionary by using the `del` keyword. The formal syntax is `del dictionary[key]`. If the key does not exist within the dictionary, you will raise a `KeyError`.

```
>>> a_dict = {"name": "Georgia Tech", "city": "Atlanta", "state":
"Georgia"}
>>> del a_dict["state"]
>>> a_dict
{"name": "Georgia Tech", "city": "Atlanta"}
>>> del a_dict["mascot"]
KeyError: "mascot"
```

Dictionary Methods

Method	Usage
.keys()	Returns a dict_keys object containing the dictionary's keys
.values()	Returns a dict_values object containing the dictionary's values
.items()	Returns a dict_items object containing the dictionary's keys and values

Sets

A set in Python is an unordered collection of items. Every element within the set is unique and must be immutable. Thus, this means you cannot place lists, dictionaries, and other sets within sets. Sets are denoted by placing elements inside {}, where each element within the set is separated by a comma, or by using the built-in `set()` function. To create an empty set, we use the `set()` function without passing in any parameters.

```
>>> a_set = {1, 1, 2, 3, "a", "a", "b"}
>>> a_set
{1, 2, 3, 'a', 'b'}
```

```
>>> a_list = [3, 4, "c", "d"]
>>> b_set = set(a_list)
>>> b_set
{3, 4, 'c', 'd'}
```

Adding and Removing Elements

You can also add and remove elements from sets. To add and remove elements, you can use the `.add()` and `.remove()` methods on pre-existing sets.

To **add** a new element, you can call the `.add()` method onto your target set. `.add()` takes in only 1 parameter, mutates the original set and returns `None`.

```
>>> a = {1, 2, 3, 4, 5}
>>> a.add(6)
>>> a
{1, 2, 3, 4, 5, 6}
```

To remove a pre-existing element from a set, you can call the `.remove()` method onto your target set. `.remove()` take in only 1 parameter, mutate the original set and return `None`.

```
>>> a = {1, 2, 3, 4, 5, 6}
>>> a.remove(6)
>>> a
{1, 2, 3, 4, 5}
```

Operations

To perform **unions** between sets, you can use the `|` operand.

```
>>> a = {0, 2, 4, 6, 8}
>>> b = {1, 2, 3, 4, 5}
>>> a | b
{0, 1, 2, 3, 4, 5, 6, 8}
```

To perform **intersections** between sets, you can use the `&` operand.

```
>>> a = {0, 2, 4, 6, 8}
>>> b = {1, 2, 3, 4, 5}
>>> a & b
{2, 4}
```

To perform **differences** between sets, you can use the `-` operand.

```
>>> a = {0, 2, 4, 6, 8}
>>> b = {1, 2, 3, 4, 5}
>>> a - b
{0, 6, 8}
```

To perform **symmetric difference** between sets, you can use the `^` operand.

```
>>> a = {0, 2, 4, 6, 8}
>>> b = {1, 2, 3, 4, 5}
>>> a ^ b
{0, 1, 3, 5, 6, 8}
```

Comprehensions

List Comprehensions

List comprehensions are used to make lists in one line of code in the form:

```
a_list = [expression for item in iterable if condition]
```

where the conditional statement is optional. This might look like

```
>>> list_1 = [i**2 for i in range(10)]
>>> list_1
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> list_2 = [i**2 for i in range(10) if i % 2 == 0]
>>> list_2
[0, 4, 16, 36, 64]
>>> list_3 = [i for i in range(10) if i % 2 == 0]
>>> list_3
[0, 2, 4, 6, 8]
```

Notice in the example above how our iterable is a range of numbers. We could also iterate through a string, list, tuple, or other iterable. Notice also how there's a conditional statement in the second and third comprehensions, but not in the first comprehension. Conditional statements aren't required, so you do not always have to use one if it can be solved without one. Likewise, your "expression" could just be "i," as you don't need to change the values from your iterable as you add it to your new list, which is seen more explicitly below.

```
>>> TAs = ["Manny", "Bohong", "Adam", "Grace", "Sydney", "Maggie"]
>>> newTAs = [name for name in TAs if "m" in name.lower()]
>>> newTAs
['Manny', 'Adam', 'Maggie']
```

In the example above we didn't use an "expression" for our name, we kept the names as they were in the initial list. In this comprehension, we iterated through the list called "TAs" and added each name to a new list ("newTAs") if that name had a "m" in it. It's important to recognize that list comprehensions generate new lists, so you should assign your comprehension to a variable. Remember that you're iterating through something (range of numbers, list, tuple, string, etc.) and adding what you want from your initial iterable (in the above case, a list called "TAs") to a *new list* if a condition is met. In the above case, the new list is called "newTAs" and the condition is "if 'm' in name.lower()." You're not changing the list called "TAs" in any way by creating "newTAs."

Dictionary Comprehensions

Dictionary comprehensions are used to make dictionaries in one line of code in the form

```
a_dict = {key:value for item in iterable if condition}
```

The conditional statement is also optional in dictionary comprehensions. Some examples of dictionary comprehensions include

```
>>> dict_1 = {i:i**2 for i in range(10)}
>>> dict_1
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
>>> dict_2 = {i:i**2 for i in range(10) if i % 2 == 0}
>>> dict_2
{0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
>>> dict_3 = {i:i*2 for i in "hippopotamus" if i in "aeiou"}
>>> dict_3
{'i': 'ii', 'o': 'oo', 'a': 'aa', 'u': 'uu'}
```

Just like in list comprehensions, our iterable can be a range of numbers, list, tuple, string, or other iterable. The conditional statement is optional, as seen in dict_1 and dict_2. It's important to recognize that the dictionary you're creating is a normal dictionary, which means there cannot be any duplicate keys. In dict_3, we iterate through the word "hippopotamus" and add each vowel from "hippopotamus" as a key mapped to the same vowel*2 (remember, multiplying strings works like "abc"*3 = "abcabcabc"). You'll notice that "o" is a duplicate vowel in "hippopotamus," but "o" only appears once as a key in dict_3. It's important to note that the vowels in the keys of dict_3 are the last occurrence of the vowels in "hippopotamus," which are shown in red above. When creating a dictionary, only the most recent key : value pair of the same "key" will appear in the dictionary.

Another useful format for creating a dictionary comprehension includes the use of zip.

```
a_dict = {key:value for key, value in zip(iterable1, iterable2)}
```

In the format above, we zip together two iterables in our dictionary comprehension, and access each element of the zip as "key, value." This is seen more explicitly below

```
>>> TAs = ["Manny", "Bohong", "Adam", "Grace", "Sydney", "Maggie"]
>>> fav_color = ["red", "green", "blue", "yellow", "blue", "pink"]
>>> dict_4 = {name:color for name, color in zip(TAs, fav_color)}
>>> dict_4
{'Manny': 'red', 'Bohong': 'green', 'Adam': 'blue', 'Grace': 'yellow', 'Sydney': 'blue', 'Maggie': 'pink'}
```

In the event that the iterables within the zip are of different length, the zip will stop after the shorter iterable ends.

```
>>> TAs = ["Manny", "Bohong", "Adam", "Grace", "Sydney", "Maggie"]
>>> fav_color = ["red", "green", "blue"]
```



```
>>> dict_5 = {name:color for name, color in zip(TAs, fav_color)}
>>> dict_5
{'Manny': 'red', 'Bohong': 'green', 'Adam': 'blue'}
```

You can also use the form

```
a_dict = {key:value for key, value in iterable if condition}
```

If the iterable is something like a list of tuples (of two elements) or a tuple of lists (of two elements). This might look like

```
>>> list_of_tups = [(1, "a"), (2, "b"), (3, "c"), (4, "d"), (5, "e"),
(6, "f")]
>>> tup_of_lists = ([1, "a"], [2, "b"], [3, "c"], [4, "d"], [5,
"e"], [6, "f"] )
>>> dict_6 = {num:letter for num,letter in list_of_tups if num%2== 0}
{2: 'b', 4: 'd', 6: 'f'}
>>> dict_7 = {num:letter for num,letter in tup_of_lists if num%2== 0}
{2: 'b', 4: 'd', 6: 'f'}
```

The dictionaries above do the same thing, but one iterates through a list of tuples and the other iterates through a tuple of lists. Remember that conditional statements will assess if the condition is met for each element in your iterable. In dict_6, the condition is assessing if the num is even for each tuple in the list of tuples, and dict_7 checks that the num is even for each list in the tuple of lists. Remember that, like in a list comprehension, a dictionary comprehension iterates through an iterable and adds a new key:value pair to a dictionary if a condition is met. This means the dictionary comprehension must be assigned to a new variable, and the dictionary will only keep the most recent key:value pair if the keys are the same.

```
>>> list_of_tups2 = [(1, "a"), (2, "b"), (3, "c"), (4, "d"), (3,
"e")]
>>> dict_8 = {num:letter for num,letter in list_of_tups2}
>>> dict_8
{1: 'a', 2: 'b', 3: 'e', 4: 'd'}
```

In the above example, dict_8[3] = "e" not "c," because (3, "e") came after (3, "c") in our iterable.

Casting to Change Data Types

In Python, data types are fundamental to how the language operates. Occasionally, you'll encounter situations where you need to convert data from one type to another. This process is known as "casting," and it allows you to manipulate and combine different data types seamlessly. Python offers a set of built-in functions to facilitate casting, ensuring your code maintains accuracy and consistency.

Implicit Type Conversion:

Python handles certain type conversions automatically when you perform operations involving different data types. For instance, if you add an integer and a floating-point number, Python will automatically convert the integer to a float for the operation. This is known as implicit type conversion.

```
integer_number = 5
floating_point_number = 3.14

result = integer_number + floating_point_number
print(result) # Outputs: 8.14
```

Explicit Type Conversion (Casting):

Explicit type conversion, or casting, gives you control over the conversion process. Python provides several built-in functions for explicit type conversion:

- `int()`: Converts a value to an integer.
- `float()`: Converts a value to a floating-point number.
- `str()`: Converts a value to a string.
- `set()`: Convert a value to a set.
- `list()`: Converts a value to a list.
- `tuple()`: Converts a value to a tuple.
- `dict()`: Converts a value to a dictionary.
- `bool()`: Converts a value to a boolean.

```
# Examples of explicit type conversion
float_number = 3.75
int_number = int(float_number) # Converts float to integer
string_number = str(int_number) # Converts integer to string
list_from_string = list(string_number) # Converts string to list

print(int_number, string_number, list_from_string)
```

Handling Potential Loss of Information:

When casting between certain data types, there might be a potential loss of information. For example, converting a floating-point number to an integer will truncate the decimal part. It's important to be aware of this possibility and consider the implications for your code.

Error Handling with Casting:

Casting between incompatible types can result in errors. For instance, converting a non-numeric string to an integer will raise a `ValueError`. To avoid these errors, you can use `try` and `except` blocks.

```
string_value = "Hello"
try:
    int_value = int(string_value)
except ValueError:
    print("Conversion failed. The input is not a valid integer.")
```

Casting plays a pivotal role in ensuring your Python code behaves as intended, especially when you're working with diverse data types. By mastering implicit and explicit type conversion, you'll have the tools to manipulate data effectively, handle different scenarios, and create code that's both efficient and robust.

Modules

Unlock the true potential of Python programming with modules! In the world of coding, modules are like specialized toolkits that bring extra functionality to your projects. Whether you're a beginner or an experienced coder, harnessing the power of modules can make you code more efficient, organized, and dynamic.

To get started, think of a module as a collection of pre-written code that you can easily integrate into your own programs. These modules contain functions, classes, and variables designed to perform specific tasks, from handling math operations to connecting with databases and even creating stunning graphical interfaces.

Using a module is a breeze. Begin by importing the desired module into your script using the `import` keyword, like so:

```
import module_name
```

Once imported, you can access the functions and features within the module using dot notation, like this:

```
module_name.function_name(arguments)
```

For instance, the popular `math` module offers a treasure trove of mathematical functions:

```
import math

result = math.sqrt(25)
print(result)  # Outputs: 5.0
```

In addition to built-in modules, you can also create your own custom modules, encapsulating your code's functionality for reuse across projects. This modular approach enhances code readability, promotes collaboration, and saves you time by avoiding code duplication.

Modules vs Scripts

There are two ways you can run Python code, one as a module and the other as a script. Now, what is the difference between a module and a script in Python? A module is generally a library that is intended to be imported by other code to use, whereas a script is a file containing code intended to be executed. In simpler terms, when we run any `.py` file through the Command Prompt / Terminal, this is considered as running the `.py` as a script. If we decided to import that same `.py` file directly into a shell such as `iPython`, this is considered as running the `.py` file as a module. Note that any Python `.py` file can be executed both as a module and as a script.

```
if __name__ == "__main__"
```

So why do we care whether we should run Python files as a script or a module?

The difference is how the Python interpreter reads the source file and defines a few special and global variables. Consider both of the following cases. If a Python file is executed as a script, the Python interpreter sets the special `__name__` variable of that file to the value of `"__main__"`. However, if a Python file is imported as a module, the Python interpreter sets the

special `__name__` variable of that file to the module's name, of the file name excluding the `.py` extension. The `__name__` is then accessible as a global variable. Given this information, we can then utilize a special if statement known as `if __name__ == "__main__"`. This allows all of the code placed within the if statement block to be executed **if and only if** the Python file is executed as a script. So when do we use `if __name__ == "__main__"`? If you are designing Python files to be imported as modules or testing different functions you have created, you will have to comment out all function calls before importing the file as a module. Instead, you can place those function calls within the `if __name__ == "__main__"` statement and have them execute only when you execute the file as a script. For example, when testing Participation Exercises or Homeworks, you will most likely be using function calls to test out your functions before submitting to GradeScope. It can be a hassle to go through your file and comment out all of your function calls, so placing all of your function calls and test cases within the `if __name__ == "__main__"` is more desirable. In addition to setting test cases to run only when executed as a script, there are times where we only want certain code to execute as a script. For example, creating GUIs using PyQt5 utilizes the `if __name__ == "__main__"` conditional for its functionality.

Example of running .py file as Module / Script

Here is an example of using `if __name__ == "__main__":` and running the file named `module_vs_script.py` as a script / module:

`module_vs_script.py`

```
print("__name__ is equal to: " + __name__)

if __name__ == "__main__":
    print("File executed as a script")
else:
    print("File executed as a module")
```

Running the Python file `module_vs_script.py` as a **module** using `iPython`:

```
>>> import module_vs_script
"File executed as a module"
"__name__ is equal to: module_vs_script"
```

Running the Python file `module_vs_script.py` as a **script** through `Command Prompt / Terminal`:

```
>>> python module_vs_script.py
"File executed as a script"
"__name__ is equal to: __main__"
```

Try and Except

The `try-except` statement in Python is used to catch and handle exceptions (errors) in your code. The code inside the `"try"` block is executed, and if an exception occurs, the code inside the `"except"` block is executed. This allows you to gracefully handle errors and avoid program crashes. Syntax:

```
try:
    # code that may raise an exception
except ExceptionType:
    # code to handle the exception
```

As an example:

```
try:
    result = x / y
except ZeroDivisionError:
    print("division by zero!")
```

You can also do an overarching “Except” and not specify an error. This will catch all possible errors.

After the except statement you can choose to include a “Finally” block, this block will ALWAYS run, no matter if there is an error or not

References

- Python Tutorials:
 - <https://www.geeksforgeeks.org/python-tutorial/>
 - <https://docs.python.org/3/tutorial/index.html>
- Python Documentation:
 - <https://docs.python.org/3/>